

20 Exercícios Comentados de Java

Do básico ao exercício integrado — variáveis, entrada de dados, condicionais, laços, ArrayList, arrays e HashMap.

Como estudar: tente resolver cada exercício sozinho antes de consultar a solução. Depois, compile e execute o código. Os comentários destacam as diferenças importantes em relação ao Python.

1. Variáveis e operações básicas

Enunciado: Crie um programa que receba o nome de uma pessoa, sua idade e sua altura. Depois, mostre esses dados e calcule quantos anos ela terá daqui a 10 anos.

Solução:

```
import java.util.Scanner;

public class Ex01 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Nome: ");
        String nome = sc.nextLine();

        System.out.print("Idade: ");
        int idade = sc.nextInt();

        System.out.print("Altura (m): ");
        double altura = sc.nextDouble();

        System.out.println("Nome: " + nome);
        System.out.println("Idade: " + idade);
        System.out.println("Altura: " + altura);
        System.out.println("Daqui a 10 anos: " + (idade + 10));

        sc.close();
    }
}
```

Comentário: Em Java, precisamos declarar o tipo de cada variável. String guarda texto, int números inteiros e double números decimais. A classe Scanner permite ler dados digitados pelo usuário.

2. Entrada de dados e média

Enunciado: Leia três notas de um aluno e calcule a média aritmética. Mostre a média com duas casas decimais.

Solução:

```
import java.util.Scanner;

public class Ex02 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Nota 1: ");
        double nota1 = sc.nextDouble();

        System.out.print("Nota 2: ");
        double nota2 = sc.nextDouble();

        System.out.print("Nota 3: ");
        double nota3 = sc.nextDouble();

        double media = (nota1 + nota2 + nota3) / 3;

        System.out.printf("Média: %.2f\n", media);

        sc.close();
    }
}
```

Comentário: O método `nextDouble()` lê números decimais. A expressão calcula a média e `printf` com `%.2f` mostra duas casas decimais.

3. Condicional simples — número positivo

Enunciado: Leia um número e informe somente se ele é positivo.

Solução:

```
import java.util.Scanner;

public class Ex03 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Digite um número: ");
        double numero = sc.nextDouble();

        if (numero > 0) {
            System.out.println("O número é positivo.");
        }

        sc.close();
    }
}
```

Comentário: O `if` simples executa seu bloco somente quando a condição entre parênteses for verdadeira. As chaves delimitam o bloco de código associado ao `if`.

4. if, else if e else — situação do aluno

Enunciado: Leia a média de um aluno. Mostre “Aprovado” para média maior ou igual a 6, “Recuperação” para média entre 4 e 5,99 e “Reprovado” para média menor que 4.

Solução:

```
import java.util.Scanner;

public class Ex04 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Média: ");
        double media = sc.nextDouble();

        if (media >= 6) {
            System.out.println("Aprovado");
        } else if (media >= 4) {
            System.out.println("Recuperação");
        } else {
            System.out.println("Reprovado");
        }

        sc.close();
    }
}
```

Comentário: Em Java, a estrutura equivalente ao elif de Python é else if. O else trata todos os casos que não satisfizeram as condições anteriores.

5. Condicionais aninhadas — classificação de idade

Enunciado: Leia a idade de uma pessoa. Se ela tiver 18 anos ou mais, diga que é adulta e, dentro desse caso, informe se é maior de 60. Para menores de 18, diga que é menor de idade.

Solução:

```
import java.util.Scanner;

public class Ex05 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Idade: ");
        int idade = sc.nextInt();

        if (idade >= 18) {
            System.out.println("Maior de idade.");

            if (idade >= 60) {
                System.out.println("Também está na faixa de 60 anos ou mais.");
            }
        } else {
            System.out.println("Menor de idade.");
        }

        sc.close();
    }
}
```

Comentário: O segundo if está dentro do primeiro, formando uma estrutura condicional aninhada. A segunda condição só é verificada quando a pessoa já foi classificada como maior de idade.

6. for simples — tabuada

Enunciado: Leia um número inteiro e mostre sua tabuada de 1 a 10 usando for.

Solução:

```
import java.util.Scanner;

public class Ex06 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Número: ");
        int numero = sc.nextInt();

        for (int i = 1; i <= 10; i++) {
            System.out.println(numero + " x " + i + " = " + (numero * i));
        }

        sc.close();
    }
}
```

Comentário: O for de Java reúne inicialização, condição e atualização. i começa em 1, continua enquanto for menor ou igual a 10 e aumenta de um em um com i++.

7. while simples — senha

Enunciado: Peça uma senha ao usuário. Enquanto a senha estiver incorreta, continue pedindo. Quando estiver correta, mostre uma mensagem de acesso liberado.

Solução:

```
import java.util.Scanner;

public class Ex07 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String senhaCorreta = "java123";
        System.out.print("Digite a senha: ");
        String senha = sc.nextLine();

        while (!senha.equals(senhaCorreta)) {
            System.out.println("Senha incorreta.");
            System.out.print("Tente novamente: ");
            senha = sc.nextLine();
        }

        System.out.println("Acesso liberado!");

        sc.close();
    }
}
```

Comentário: Para comparar o conteúdo de Strings em Java, usamos equals(), e não ==. O operador ! nega a condição. O while continua enquanto a senha for diferente da correta.

8. for aninhado — matriz de números

Enunciado: Mostre uma matriz 3 × 4 contendo os números de 1 a 12, usando um for dentro de outro for.

Solução:

```
public class Ex08 {
    public static void main(String[] args) {
        int numero = 1;

        for (int linha = 0; linha < 3; linha++) {
            for (int coluna = 0; coluna < 4; coluna++) {
                System.out.print(numero + " ");
                numero++;
            }

            System.out.println();
        }
    }
}
```

Comentário: O for externo controla as linhas e o interno as colunas. Para cada linha, o laço interno executa quatro vezes. print() mantém os números na mesma linha; println() pula para a próxima.

9. while aninhado — contador de combinações

Enunciado: Use dois while aninhados para imprimir todas as combinações de dois números, sendo o primeiro de 1 a 3 e o segundo de 1 a 2.

Solução:

```
public class Ex09 {
    public static void main(String[] args) {
        int a = 1;

        while (a <= 3) {
            int b = 1;

            while (b <= 2) {
                System.out.println("(" + a + ", " + b + ")");
                b++;
            }

            a++;
        }
    }
}
```

Comentário: Para cada valor de a, o segundo while começa novamente com b = 1. Os incrementos são essenciais para evitar um laço infinito.

10. ArrayList — adicionar, remover e percorrer

Enunciado: Crie uma lista de frutas. Adicione uma fruta, remova outra e depois percorra a lista mostrando cada item.

Solução:

```
import java.util.ArrayList;

public class Ex10 {
    public static void main(String[] args) {
        ArrayList<String> frutas = new ArrayList<>();

        frutas.add("maçã");
        frutas.add("banana");
        frutas.add("laranja");

        frutas.add("uva");           // adiciona
        frutas.remove("banana");     // remove pelo valor

        for (String fruta : frutas) {
            System.out.println(fruta);
        }
    }
}
```

Comentário: Em Java, a estrutura mais próxima da lista dinâmica de Python é ArrayList. add() adiciona elementos e remove() remove um elemento. O for-each percorre todos os itens.

11. ArrayList — notas e maior valor

Enunciado: Leia cinco notas, coloque-as em uma lista e depois mostre todas as notas, a maior, a menor e a média.

Solução:

```
import java.util.ArrayList;
import java.util.Scanner;

public class Ex11 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Double> notas = new ArrayList<>();

        for (int i = 0; i < 5; i++) {
            System.out.print("Nota " + (i + 1) + ": ");
            notas.add(sc.nextDouble());
        }

        double maior = notas.get(0);
        double menor = notas.get(0);
        double soma = 0;

        for (double nota : notas) {
            if (nota > maior) {
                maior = nota;
            }

            if (nota < menor) {
                menor = nota;
            }

            soma += nota;
        }

        System.out.println("Notas: " + notas);
        System.out.println("Maior: " + maior);
        System.out.println("Menor: " + menor);
        System.out.println("Média: " + (soma / notas.size()));

        sc.close();
    }
}
```

Comentário: ArrayList<Double> armazena números decimais. Diferentemente de Python, aqui não usamos max() e min() diretamente: o exercício mostra como encontrar esses valores percorrendo a lista. size() informa a quantidade de elementos.

12. Tupla — equivalente em Java

Enunciado: Java não possui uma estrutura chamada tuple equivalente à tupla de Python na biblioteca padrão. Para praticar uma coleção de dados fixos, use um array de String contendo os meses do ano. Mostre o terceiro mês e percorra todos eles.

Solução:

```
public class Ex12 {
    public static void main(String[] args) {
        final String[] meses = {
            "janeiro", "fevereiro", "março", "abril",
            "maio", "junho", "julho", "agosto",
            "setembro", "outubro", "novembro", "dezembro"
        };

        System.out.println("Terceiro mês: " + meses[2]);

        for (String mes : meses) {
            System.out.println(mes);
        }
    }
}
```

Comentário: Este exercício faz uma adaptação importante: Java não possui tuplas nativas como Python. Um array de tamanho fixo é uma alternativa simples para uma sequência cujo tamanho não muda. Como os índices começam em zero, meses[2] é o terceiro elemento.

13. HashMap — adicionar, alterar e remover

Enunciado: Crie um mapa com nome e idade. Adicione a profissão, altere a idade e remova a profissão. No final, percorra o mapa mostrando chave e valor.

Solução:

```
import java.util.HashMap;
import java.util.Map;

public class Ex13 {
    public static void main(String[] args) {
        HashMap<String, String> pessoa = new HashMap<>();

        pessoa.put("nome", "Carlos");
        pessoa.put("idade", "30");

        pessoa.put("profissao", "Químico"); // adicionar
        pessoa.put("idade", "31");          // alterar
        pessoa.remove("profissao");          // remover

        for (Map.Entry<String, String> entrada : pessoa.entrySet()) {
            System.out.println(entrada.getKey() + ": " + entrada.getValue());
        }
    }
}
```

Comentário: O equivalente mais próximo do dicionário de Python é o HashMap. put() adiciona ou altera uma chave, remove() exclui uma entrada e entrySet() permite percorrer chave e valor.

14. HashMap — cadastro de produtos

Enunciado: Crie um mapa de produtos e preços. Permita ao usuário digitar o nome de um produto para consultar seu preço. Trate o caso em que o produto não existe.

Solução:

```
import java.util.HashMap;
import java.util.Scanner;

public class Ex14 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        HashMap<String, Double> produtos = new HashMap<>();

        produtos.put("arroz", 25.90);
        produtos.put("feijão", 8.50);
        produtos.put("café", 18.00);

        System.out.print("Produto: ");
        String produto = sc.nextLine().toLowerCase();

        if (produtos.containsKey(produto)) {
            System.out.printf("Preço: R$ %.2f%n", produtos.get(produto));
        } else {
            System.out.println("Produto não cadastrado.");
        }

        sc.close();
    }
}
```

Comentário: `containsKey()` verifica se uma chave existe. `get()` recupera seu valor. O método `toLowerCase()` ajuda a padronizar a entrada do usuário.

15. ArrayList + condicionais — separar pares e ímpares

Enunciado: Leia dez números inteiros e coloque os pares em uma lista e os ímpares em outra. No final, mostre as duas listas.

Solução:

```
import java.util.ArrayList;
import java.util.Scanner;

public class Ex15 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        ArrayList<Integer> pares = new ArrayList<>();
        ArrayList<Integer> impares = new ArrayList<>();

        for (int i = 0; i < 10; i++) {
            System.out.print("Número " + (i + 1) + ": ");
            int numero = sc.nextInt();

            if (numero % 2 == 0) {
                pares.add(numero);
            } else {
                impares.add(numero);
            }
        }

        System.out.println("Pares: " + pares);
        System.out.println("Ímpares: " + impares);

        sc.close();
    }
}
```

Comentário: O operador % calcula o resto da divisão. Se o resto da divisão por 2 for zero, o número é par. O exercício combina for, if/else e ArrayList.

16. Fibonacci com for

Enunciado: Leia a quantidade de termos desejada e mostre a sequência de Fibonacci usando for. Considere que a sequência começa em 0, 1, 1, 2...

Solução:

```
import java.util.Scanner;

public class Ex16 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Quantidade de termos: ");
        int n = sc.nextInt();

        long a = 0;
        long b = 1;

        for (int i = 0; i < n; i++) {
            System.out.print(a + " ");
            long proximo = a + b;
            a = b;
            b = proximo;
        }

        System.out.println();
        sc.close();
    }
}
```

Comentário: a e b representam dois termos consecutivos. Em Java, usamos uma variável auxiliar proximo para guardar a soma antes de atualizar a e b. O tipo long permite números maiores que int.

17. Fibonacci com while

Enunciado: Resolva o mesmo problema anterior, mas agora usando while em vez de for.

Solução:

```
import java.util.Scanner;

public class Ex17 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Quantidade de termos: ");
        int n = sc.nextInt();

        long a = 0;
        long b = 1;
        int contador = 0;

        while (contador < n) {
            System.out.print(a + " ");

            long proximo = a + b;
            a = b;
            b = proximo;

            contador++;
        }

        System.out.println();
        sc.close();
    }
}
```

Comentário: No while, precisamos controlar explicitamente o número de repetições. A variável contador aumenta a cada ciclo até atingir n.

18. ArrayList + while — cadastro até sair

Enunciado: Peça nomes ao usuário repetidamente. Guarde-os em uma lista até que ele digite “fim”. Depois, percorra e mostre todos os nomes cadastrados.

Solução:

```
import java.util.ArrayList;
import java.util.Scanner;

public class Ex18 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<String> nomes = new ArrayList<>();

        while (true) {
            System.out.print("Nome (ou 'fim'): ");
            String nome = sc.nextLine();

            if (nome.equalsIgnoreCase("fim")) {
                break;
            }

            nomes.add(nome);
        }

        System.out.println("Nomes cadastrados:");

        for (String nome : nomes) {
            System.out.println("- " + nome);
        }

        sc.close();
    }
}
```

Comentário: while (true) cria um laço que só termina quando encontramos break. equalsIgnoreCase() permite aceitar “fim”, “FIM”, “Fim” etc. Os nomes são armazenados em um ArrayList.

19. HashMap + for — contagem de palavras

Enunciado: Leia uma frase e conte quantas vezes cada palavra aparece. Use um HashMap para armazenar as contagens.

Solução:

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class Ex19 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Digite uma frase: ");
        String frase = sc.nextLine().toLowerCase();

        String[] palavras = frase.split("\\s+");
        HashMap<String, Integer> contagem = new HashMap<>();

        for (String palavra : palavras) {
            if (contagem.containsKey(palavra)) {
                contagem.put(palavra, contagem.get(palavra) + 1);
            } else {
                contagem.put(palavra, 1);
            }
        }

        for (Map.Entry<String, Integer> entrada : contagem.entrySet()) {
            System.out.println(entrada.getKey() + ": " + entrada.getValue());
        }

        sc.close();
    }
}
```

Comentário: `split("\\s+")` separa a frase pelos espaços em branco. Para cada palavra, verificamos se ela já existe no mapa. Se existir, incrementamos a contagem; caso contrário, criamos a entrada com valor 1.

20. Exercício integrado — sistema de notas

Enunciado: Faça um pequeno sistema que cadastre alunos e suas notas em um HashMap. O programa deve permitir cadastrar, consultar e remover alunos até que o usuário escolha sair. Ao cadastrar, armazene uma lista de três notas para cada aluno e mostre a média na consulta.

Solução:

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Scanner;

public class Ex20 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        HashMap<String, ArrayList<Double>> alunos = new HashMap<>();

        while (true) {
            System.out.println("\n1 - Cadastrar");
            System.out.println("2 - Consultar");
            System.out.println("3 - Remover");
            System.out.println("4 - Sair");
            System.out.print("Opção: ");

            int opcao = sc.nextInt();
            sc.nextLine(); // consome o ENTER

            if (opcao == 1) {
                System.out.print("Nome do aluno: ");
                String nome = sc.nextLine();

                ArrayList<Double> notas = new ArrayList<>();

                for (int i = 0; i < 3; i++) {
                    System.out.print("Nota " + (i + 1) + ": ");
                    notas.add(sc.nextDouble());
                }
                sc.nextLine();

                alunos.put(nome, notas);
                System.out.println("Aluno cadastrado.");
            } else if (opcao == 2) {
                System.out.print("Nome do aluno: ");
                String nome = sc.nextLine();

                if (alunos.containsKey(nome)) {
                    ArrayList<Double> notas = alunos.get(nome);
                    double soma = 0;

                    for (double nota : notas) {
                        soma += nota;
                    }

                    double media = soma / notas.size();

                    System.out.println("Notas: " + notas);
                    System.out.printf("Média: %.2f%n", media);
                } else {
                    System.out.println("Aluno não encontrado.");
                }
            } else if (opcao == 3) {
                System.out.print("Nome do aluno: ");
                String nome = sc.nextLine();

                if (alunos.containsKey(nome)) {
```

```

        alunos.remove(nome);
        System.out.println("Aluno removido.");
    } else {
        System.out.println("Aluno não encontrado.");
    }

    } else if (opcao == 4) {
        System.out.println("Programa encerrado.");
        break;

    } else {
        System.out.println("Opção inválida.");
    }
}

sc.close();
}
}

```

Comentário: Este exercício reúne praticamente tudo da lista: variáveis, Scanner, if/else if/else, condicionais aninhadas, while, for, ArrayList e HashMap. O mapa usa o nome do aluno como chave e uma lista de três notas como valor. Assim, temos uma estrutura de dados dentro de outra.